# Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

**Thème principal :** Variables, mémoire RAM, affectation, structures conditionnelles (Si/Sinon) et itératives (Pour/TantQue)

**Synthèse exécutive :** Ce cours pose les fondations fondamentales du développement logiciel et de la pensée algorithmique en BUT Informatique. Il résout le problème central de la traduction d'un processus logique du monde réel en une suite formelle, séquentielle et déterministe d'instructions exécutables par un processeur. Le cours détaille la modélisation de la mémoire vive (RAM) à travers le quadruplet caractérisant une variable (Nom, Type, Valeur, Adresse), la mécanique de l'affectation destructive, l'aiguillage du flux d'exécution séquentiel à l'aide de prédicats booléens et de structures conditionnelles (Si / Alors / Sinon), ainsi que l'automatisation de traitements répétitifs au moyen de boucles bornées (Pour) et non bornées (TantQue).

## Mémorisation et Manipulation des Données : Les Variables et la Mémoire RAM

En informatique, le concept de variable se distingue radicalement de sa définition mathématique statique. Il ne s'agit pas d'une inconnue abstraite dans une équation, mais d'une case mémoire physique concrète située dans la RAM de l'ordinateur. Une variable est obligatoirement caractérisée par un quadruplet : son Nom (qui permet au développeur de la manipuler dans le code), son Type (qui indique au système le nombre d'octets à réserver et le format de codage binaire), sa Valeur (le contenu binaire à un instant t) et son Adresse mémoire physique (emplacement hexadécimal en RAM). Lors de l'exécution d'une instruction d'affectation (par exemple x ← x + 1), l'Unité Centrale de Traitement (CPU) applique un protocole en deux étapes chronologiques : premièrement, elle lit en mémoire RAM les valeurs actuelles des variables figurant à droite du symbole d'affectation et évalue l'expression globale ; deuxièmement, elle vient inscrire ce résultat final dans la case mémoire associée à la variable située à gauche. Ce processus est strictement destructif : le nouveau résultat écrase définitivement l'ancienne valeur. Le piège majeur en partiel consiste à utiliser une variable sans l'avoir initialisée. Si un algorithme cherche à lire une variable non initialisée dans un calcul, le système va récupérer les bits résiduels restés en mémoire vive (la 'valeur poubelle'), engendrant des calculs faux ou des comportements totalement imprévisibles du programme.

- Variable : Emplacement mémoire étiqueté réservé dans la mémoire vive (RAM) destiné à conserver une valeur modifiable pendant l'exécution du programme.
- Quadruplet d'une variable : Ensemble indissociable constitué par son Nom (identificateur), son Type (format d'encodage et taille en octets), sa Valeur actuelle et son Adresse physique en RAM.
- Typage formel : Spécification explicite de la nature de la donnée (Entier, Réel, Chaîne de caractères, Booléen) déterminant la quantité de mémoire allouée et les opérations autorisées.
- Affectation (← ou =) : Opération fondamentale destructive d'écriture qui évalue l'expression située à droite du symbole et écrase irrémédiablement la valeur stockée à gauche.
- Valeur poubelle : Donnée binaire résiduelle non nettoyée présente en RAM, lue lorsqu'une variable est exploitée sans avoir été préalablement initialisée.

***Cas concret enseignant :*** *La métaphore de l'armoire d'archivage à tiroirs : Imaginez la mémoire RAM comme une grande armoire d'archivage contenant des tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à dimensionner sa profondeur selon le type ('Entier'). L'instruction d'affectation 'solde_compte ← 150' consiste à ouvrir ce tiroir précis, à jeter au recyclage le morceau de papier qui s'y trouvait auparavant (l'ancienne valeur), et à y déposer une nouvelle feuille portant l'inscription 150.*

## Prise de Décision : Les Structures Conditionnelles (Si / Sinon) et Logique Booléenne

Par défaut, un ordinateur exécute un programme de manière séquentielle, de la première à la dernière instruction, ligne après ligne. Pour doter l'algorithme d'un comportement adaptatif et intelligent, on introduit des structures conditionnelles. Lorsqu'un bloc 'Si (Condition) Alors [Bloc A] Sinon [Bloc B]' est rencontré, le processeur commence par évaluer l'expression booléenne. Si le prédicat donne la valeur VRAI, l'ordinateur exécute exclusivement le [Bloc A], puis saute par-dessus le [Bloc B] pour reprendre le flux normal du programme. Si la condition donne la valeur FAUX, le [Bloc A] est ignoré et seul le [Bloc B] (s'il existe) est exécuté. Pour formaliser des critères décisionnels complexes, la maîtrise des tables de vérité est requise : l'opérateur ET (AND) exige que l'ensemble des sous-conditions soient simultanément vraies pour valider le test global ; l'opérateur OU (OR) requiert qu'au moins une des sous-conditions soit vraie ; l'opérateur NON (NOT) inverse la valeur de vérité d'une expression.

- Structure d'Alternative (Si / Alors / Sinon) : Aiguillage logique détournant le flux d'exécution séquentiel selon la valeur de vérité d'une condition.
- Expression Booléenne (Prédicat) : Proposition logique évaluée par le processeur qui ne peut prendre que deux états strictement binaires : VRAI (True) ou FAUX (False).
- Opérateurs Logiques (ET, OU, NON) : Connecteurs booléens permettant de composer des conditions complexes régies par leurs tables de vérité respectives.

***Cas concret enseignant :*** *Le cas réel du Distributeur Automatique de Billets (DAB) : Avant d'expulser les billets de banque, l'automate évalue une condition composée : SI (solde_client >= montant_demandé) ET (billets_disponibles_dans_automate >= montant_demandé). Si cette expression globale s'évalue à VRAI, le paiement est validé et les billets sont délivrés ; SINON, l'opération est refusée, un message d'erreur est affiché sur l'écran et la carte bancaire est restituée à l'usager.*

## Répétition et Automatisation : Les Structures Itératives (Boucles Pour et TantQue)

La véritable puissance des systèmes informatiques réside dans leur capacité à exécuter des séries d'instructions répétitives à très haute vitesse et sans altération. Deux structures itératives principales sont utilisées en algorithmique : 1. La boucle POUR (déterministe) : Elle est privilégiée lorsque le nombre d'itérations est connu avant d'entrer dans la boucle. Elle intègre nativement l'initialisation d'une variable compteur, le test de borne supérieure et l'incrémentation systématique à chaque itération (ex: traiter les N éléments d'un tableau). 2. La boucle TANT QUE (indéterminée) : Elle s'impose lorsque le nombre de répétitions dépend d'un événement dynamique au cours du programme (ex: valider la saisie d'un mot de passe utilisateur). La condition est évaluée AVANT chaque passage. Si elle s'avère vraie, le corps de la boucle s'exécute ; si elle est fausse dès le départ, le corps n'est jamais exécuté. La règle d'or d'une boucle TANT QUE est de modifier impérativement, à l'intérieur du corps de la boucle, au moins une variable engagée dans la condition de sortie. Si la condition reste indéfiniment VRAIE, l'algorithme tombe dans une boucle infinie qui bloque l'exécution et sature le processeur.

- Boucle Pour (FOR) : Structure itérative bornée et déterministe, utilisée lorsque le nombre précis de répétitions est connu avant le début de l'exécution.
- Boucle TantQue (WHILE) : Structure itérative non bornée et dynamique, répétée tant qu'une condition logique demeure VRAIE.
- Variable de Contrôle & Incrémentation : Modification explicite d'un compteur au sein du corps de la boucle pour garantir le franchissement de la condition de sortie.
- Boucle Infinie : Dysfonctionnement critique lié à l'absence d'actualisation de la condition de sortie dans une boucle TantQue, gelant le programme et monopolisant 100% des ressources CPU.

***Cas concret enseignant :*** *La métaphore des consignes de l'entraîneur d'athlétisme : Si l'entraîneur ordonne au coureur 'Fais 10 tours de piste', il s'agit d'une boucle POUR (le nombre 10 est fixé dès le départ et la fin est garantie après 10 tours). S'il ordonne 'Cours tant qu'il ne pleut pas', il s'agit d'une boucle TANT QUE (la sortie dépend d'une observation météo réévaluée à chaque tour de piste).*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Pourquoi l'instruction 'x = x + 1' (ou 'x ← x + 1') est-elle valide en programmation alors qu'elle est mathématiquement impossible ?
**Réponse du professeur :** En mathématiques, l'égalité 'x = x + 1' est une équation statique irréalisable. En informatique, le symbole '=' ou '←' représente une affectation dynamique et destructive. Le processeur évalue d'abord le membre de droite en lisant la valeur actuelle de 'x' en RAM (ex: 5), y ajoute 1 pour obtenir 6, puis va écrire le résultat 6 dans la case mémoire 'x' située à gauche, écrasant la valeur 5 précédente.

**Question étudiant :** Que se passe-t-il concrètement si j'oublie de modifier la variable de contrôle à l'intérieur du corps d'une boucle TantQue ?
**Réponse du professeur :** Si la variable de contrôle présente dans le test d'entrée n'est jamais réactualisée dans le corps de la boucle, la condition reste VRAIE à chaque itération. Le programme rentre immédiatement dans une 'boucle infinie'. L'ordinateur gèle le processus, la fenêtre ne répond plus et le processeur tourne à 100% de charge sur ce cœur jusqu'à interruption forcée de l'exécution.

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire un programme complet en C++ (selon les normes de l'IUT d'Anglet) qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (Somme = 1 + 2 + ... + N).

**Démarche de résolution :**
Étape 1 : Rédaction du cartouche d'en-tête indiquant le nom du fichier, l'auteur, la date et l'objectif pédagogique. Étape 2 : Structuration du code selon la tripartition réglementaire IUT (// CONSTANTES, // VARIABLES, // TRAITEMENTS). Étape 3 : Déclaration explicite des variables n, i et somme en type entier ('int'). Étape 4 : Saisie utilisateur du paramètre n annotée par la flèche de flot de données '// (clavier) >> saisir >> n'. Étape 5 : Initialisation explicite de 'somme = 0' avant d'entrer dans la boucle afin de vider toute valeur poubelle résiduelle en mémoire RAM. Étape 6 : Construction d'une boucle 'for' allant de i = 1 à i <= n par pas de 1, avec indentation de style Allman (accolade ouvrante sur sa propre ligne). Étape 7 : Accumulation de la valeur de i dans 'somme' à chaque tour (somme = somme + i). Étape 8 : Affichage clair du résultat final sur 'cout'.

```
/*
  Nom : calcul_somme.cpp
  Auteur : Matéi
  Date : 12/11/2026
  Objectif : Calcule la somme des N premiers entiers positifs de 1 a N
*/
#include <iostream>
using namespace std;

int main()
{
    // CONSTANTES

    // VARIABLES
    int n;     // Nombre limite saisi par l'utilisateur
    int i;     // Compteur d'iterations
    int somme; // Accumulateur de la somme

    // TRAITEMENTS
    // (clavier) >> saisir >> n
    cout << "Veuillez saisir un nombre entier positif N : ";
    cin >> n;

    // Initialisation stricte pour ecraser toute valeur poubelle en RAM
    somme = 0;

    for (i = 1; i <= n; i++)
    {
        somme = somme + i;
    }

    cout << "La somme des entiers de 1 a " << n << " est egale a : " << somme << endl;

    return 0;
}
```

**Résultat : Pour une saisie utilisateur N = 5, le suivi pas à pas en mémoire RAM donne : - Départ : somme = 0 - i = 1 → somme = 0 + 1 = 1 - i = 2 → somme = 1 + 2 = 3 - i = 3 → somme = 3 + 3 = 6 - i = 4 → somme = 6 + 4 = 10 - i = 5 → somme = 10 + 5 = 15 Le programme affiche à l'écran : 'La somme des entiers de 1 a 5 est egale a : 15'. L'exécution est déterministe et conforme aux exigences académiques de l'IUT.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quelle est la différence fondamentale de critère de choix entre une boucle POUR et une boucle TANT QUE ?**

*R : La boucle POUR s'impose lorsque le nombre d'itérations est déterministe et connu à l'avance (ex: fixer 10 tours ou parcourir N éléments). La boucle TANT QUE s'utilise lorsque le nombre de répétitions est indéterminé et dépend d'une condition dynamique évaluée au fil de l'exécution (ex: validation d'une saisie utilisateur).*

**Q : Que valent les variables A et B après l'exécution exacte de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?**

*R : Après A ← 5 et B ← 3, l'instruction A ← A + B évalue 5 + 3 = 8, donc A prend la valeur 8. L'instruction suivante B ← A - B évalue 8 - 3 = 5, donc B prend la valeur 5. Au final : A = 8 et B = 5 (c'est l'algorithme classique d'échange de deux variables sans variable temporaire).*

**Q : Pourquoi l'absence d'initialisation d'une variable d'accumulation (ex: somme ← 0) entraîne-t-elle la perte de points en partiel ?**

*R : Parce qu'en C++ et en algorithmique, une variable non initialisée contient une 'valeur poubelle' (des bits aléatoires résiduels restés en mémoire RAM). Effectuer 'somme = somme + i' sans initialiser 'somme' ajoute 'i' à un nombre inconnu, rendant le calcul faux et le comportement du programme totalement imprévisible.*